Verified Dyadic Deontic Logic and Governance Reasoning in Lean 4
(DRAFT --- March 2, 2026)

Zar* Claude Code (Opus 4.6)†

March 2, 2026

Abstract

We present a Lean 4 formalization of governance deontic reasoning, spanning from classical deontic logic through eventuality reification to constructive governance models and ethical foundations. The formalization includes: (i) a shallow semantic embedding of Carmo and Jones' Dyadic Deontic Logic (DDL+) with five semantic conditions; (ii) a proof of all 13 Carmo-Jones axiomatic characterisation theorems (CJ_3–CJ_15) as consequences of the embedding; (iii) the Deontic Traditional Scheme (DTS) with obligation as the sole primitive, derived permission and optionality, and all 12 von Wright interdefinability relations as theorems; (iv) a governance reasoning layer with Hobbs-style eventuality reification, ISO 24617-4 thematic roles, a deontic query encoder, and a three-level judgment pipeline; (v) a constructive 2-state reactive loop (`GovNormCycle`) providing a provably seriality-closed accessibility context; (vi) a temporal-deontic bridge formalizing obligation persistence under time translation and a causal-normative prohibition theorem ruling out spurious causal candidates from obligation; (vii) an abstract `ModalQueryEncoder` unifying event-calculus and deontic encoders via a shared interface; (viii) an application to Gewirth's Principle of Generic Consistency (PGC); and (ix) formal refinement theorems connecting MeTTa governance rule implementations to the Lean DTS semantics, via a shared `MeTTaLike` evaluator abstraction covering both PeTTa and HE MeTTa evaluation, with proven `let*` unfolding equivalences and kernel-checked canonical HE interpreter conformance tests.

Contents

1 Introduction **3**

*Corresponding author.
†Anthropic AI assistant. Co-authored under human direction.

2 Shallow Embedding of DDL+ **4**
2.1 Types and Semantic Foundation 4
2.2 The DDL+ Frame 4
2.3 Modal and Deontic Operators 5
2.4 Kaplanian Context and LD Validity 5

3 Carmo-Jones Axiomatic Characterisation **6**

4 Deontic Traditional Scheme **6**

5 Governance Reasoning Layer **7**
5.1 Hobbs Eventuality Reification 7
5.2 Deontic Query Encoder 8
5.3 Rexist Bridge 8
5.4 Three-Level Judgment Pipeline 8
5.5 Worked Example: Rental Obligation 9

6 GovNormCycle: Constructive Reactive Accessibility **10**

7 Temporal-Deontic Bridge **11**
7.1 Temporal Meanings 11
7.2 Temporally Uniform Obligation 11
7.3 Obligation Persistence Theorem 12
7.4 Causal-Normative Prohibition 12

8 Abstract Modal Query Encoder **13**

9 Gewirth's Principle of Generic Consistency **14**

10 MeTTa-to-Lean Refinement Theorems **14**
10.1 Architecture 14
10.2 PeTTa Refinement 15
10.3 Shared MeTTaLike Interface and Let* Equivalence 15
10.4 HE MeTTa Refinement 16
10.5 Canonical HE Spec Architecture 16
10.6 What the Refinement Proves 17

11 Related Work **18**

12 Limitations **18**

13 Future Developments: Toward Practical Governance **19**

14 Conclusion **20**

1 Introduction

Formal governance and policy reasoning require a robust logical foundation that can represent obligations, permissions, and temporal dynamics. Classical propositional deontic logic [17] provides a starting point, but real governance scenarios involve:

- *Contrary-to-duty obligations* — norms that apply precisely when a primary obligation is violated [4];
- *Eventuality reification* — treating events and states as first-class entities so that thematic roles and temporal structure can be reasoned about explicitly [10];
- *Temporal dynamics* — obligations that persist, shift, or expire as time advances; and
- *Causal spuriousness* — distinguishing genuine causal obligations from extensional correlates that lack intensional support.

This paper describes a Lean 4 formalization in the `Mettapedia` repository that addresses all four concerns. The architecture layers from abstract to concrete: Carmo-Jones DDL+ semantics → DTS algebra → governance reasoning layer → reactive accessibility model → temporal and causal bridges → Gewirth PGC application.

Contributions. The primary contributions are:

1. **Lean 4 port of the Benzmüller et al. Isabelle/HOL DDL+ embedding** [2], making it available in the Lean 4 ecosystem with Mathlib compatibility. All 13 Carmo-Jones theorems are proven from 5 semantic conditions (§2–3).
2. **A Lean 4 port of Benzmüller et al.'s Isabelle/HOL proof of Gewirth's Principle of Generic Consistency** (PGC) [3], connecting it to the DDL+ and DTS layers above (§9).
3. **An honest architectural exploration** of whether formal deontic logic can ground governance reasoning, including a negative result: simple terminating languages do *not* satisfy the seriality requirement needed for governance accessibility (§6).
4. **A temporal-deontic bridge** formalizing obligation persistence under time translation and a causal-normative prohibition theorem connecting PLN causal profiles to DTS (§7).
5. **Formal MeTTa-to-Lean refinement theorems** proving that MeTTa governance rule implementations, when interpreted through a deontic pattern decoder, correspond to the proven DTS theorems — covering PeTTa and HE MeTTa evaluators via a shared typeclass, with kernel-checked canonical HE interpreter conformance (§10).

Supporting contributions include: a DTS algebra with derived permission and optionality (§4); a governance reasoning layer with Hobbs eventuality reification, ISO 24617-4 thematic roles, and a three-level verdict pipeline (§5); an abstract `ModalQueryEncoder` unifying event-calculus and deontic query dispatch (§8); and a shared `MeTTaLike` evaluator typeclass with proven `let*` unfolding equivalences (§10).

2 Shallow Embedding of DDL+

2.1 Types and Semantic Foundation

The DDL+ embedding follows the standard *possible-worlds semantics* tradition, representing propositions as truth-sets over a world type. We adopt Kaplan's two-dimensional semantics [12] to handle context-dependent propositions ("characters").

Definition 1 (World-Proposition and Meaning)**.** *A* world-proposition *over worlds* `w` *is a predicate* `WProp w = w -> Prop`*. A* meaning *(Kaplan's character) is a context-dependent proposition* `Meaning c w = c -> w -> Prop`*.*

These correspond to Isabelle types `wo = w => bool` and `m = c => w => bool` in the source formalization [2].

2.2 The DDL+ Frame

The semantic foundation of DDL+ is a *frame* bundling two accessibility relations and a dyadic obligation relation:

```
structure DDLPlusFrame (w : Type*) where
  av : w → w → Prop   -- actual-version accessibility
  pv : w → w → Prop   -- possible-version accessibility
  ob : WProp w → WProp w → Prop  -- dyadic obligation
  sem_3a :   ∀   v,   ∃   v', av v v'      -- av is serial
  sem_4a :   ∀   v v', av v v' → pv v v'   -- av <= pv
  sem_4b :   ∀   v, pv v v                 -- pv is reflexive
  sem_5a :   ∀   X, not (ob X (fun _ => False))
  sem_5b :   ∀   X Y Z, (  ∀   v, (X v /\ Y v)  ↔  (X v /\ Z v))
      →
             (ob X Y  ↔  ob X Z)
  sem_5c :   ∀   X Y Z, (  ∃   v, X v /\ Y v /\ Z v) →
             ob X Y → ob X Z → ob X (fun v => Y v /\ Z v)
  sem_5d :   ∀   X Y Z, (  ∀   v, Y v → X v) → ob X Y →
             (  ∀   v, X v → Z v) →
             ob Z (fun v => (Z v /\ not (X v)) \/ Y v)
  sem_5e :   ∀   X Y Z, (  ∀   v, Y v → X v) → ob X Z →
             (  ∃   v, Y v /\ Z v) → ob Y Z
```

The five conditions `sem_3a`–`sem_5e` match Carmo & Jones (2002) conditions (3a), (4a), (4b), (5a)–(5e) [4], pp. 290ff., and correspond to lines 52–59 of the Isabelle source `CJDDLplus.thy`.

Remark 2. *The accessibility relation* `av` *models "open alternatives" (what can actually happen), while* `pv` *models "possible alternatives" (what could ideally happen). Condition* `sem_4a` *ensures open alternatives are a subset of possible ones, and* `sem_4b` *ensures reflexivity of* `pv`*.*

2.3 Modal and Deontic Operators

Six modal operators are defined over `DDLPlusFrame`:

```
def box_a (phi : Meaning c w) : Meaning c w :=  -- actual necessity
  fun ctx v =>   ∀   v', F.av v v' → phi ctx v'
def box_p (phi : Meaning c w) : Meaning c w :=  -- possible
    necessity
  fun ctx v =>   ∀   v', F.pv v v' → phi ctx v'
def box_S5 (phi : Meaning c w) : Meaning c w :=  -- unrestricted
  fun ctx _ =>   ∀   v, phi ctx v
```

Diamond operators are defined dually. Three deontic operators are then derived:

```
-- Conditional obligation O<phi|sigma>
def cond_obl (phi sigma : Meaning c w) : Meaning c w :=
  fun ctx _ => F.ob (sigma ctx) (phi ctx)

-- Actual obligation: ob relative to av, with violation witness
def actual_obl (phi : Meaning c w) : Meaning c w :=
  fun ctx v => F.ob (F.av v) (phi ctx)
    /\   ∃   v', F.av v v' /\ not (phi ctx v')

-- Ideal obligation: ob relative to pv, with violation witness
def ideal_obl (phi : Meaning c w) : Meaning c w :=
  fun ctx v => F.ob (F.pv v) (phi ctx)
    /\   ∃   v', F.pv v v' /\ not (phi ctx v')
```

The violation-witness condition (the $\exists v'$ clause) is standard in contrary-to-duty frameworks: an obligation is "ideal" only when there exist possible worlds where it is not satisfied.

2.4 Kaplanian Context and LD Validity

To model indexical statements, we follow Kaplan's two-dimensional structure [12]:

```
structure KaplanianContext (c w e : Type*) where
  agent : c → e    -- agent of the context
  world : c → w    -- world of the context
```

LD validity (truth in every context at its own world) is strictly weaker than classical modal validity. This distinction is proven as a theorem: `modal_implies_ld` shows that modal validity entails LD validity, but not conversely. The *a priori necessity* operator $\Box^D$ is truth at all LD contexts: `box_D phi ctx := ld_valid phi`.

3 Carmo-Jones Axiomatic Characterisation

Carmo and Jones [4] present 13 axioms (CJ_3–CJ_15) as the axiomatic characterisation of DDL+. Following Benzmüller et al. [2], we derive all 13 from the 5 semantic frame conditions, confirming the soundness of the axiomatic characterisation with respect to the possible-worlds semantics. Table 1 lists each theorem with its Lean identifier and the primary frame condition used in its proof.

Table 1: Carmo-Jones theorems proven semantically in Lean 4

Theorem	**Lean identifier**	**Key condition**
CJ_3: $\Box_p \Rightarrow \Box_a$	`CJ_3`	`sem_4a`
CJ_4: $\neg O\langle\bot\|A\rangle$	`CJ_4`	`sem_5a`
CJ_5$^-$: conjunction of obligations	`CJ_5_minus`	`sem_5c`
CJ_6: obligation closed under conjunction	`CJ_6`	`sem_5e`
CJ_7: modal equivalence preservation	`CJ_7`	`sem_5e`
CJ_8: conditional equivalence preservation	`CJ_8`	`C_6`
CJ_9a: $\Diamond_p O \Rightarrow \Box_p O$	`CJ_9a`	O is world-independent
CJ_9p: $\Diamond_a O \Rightarrow \Box_a O$	`CJ_9p`	O is world-independent
CJ_9_var_a: $O \Rightarrow \Box_a O$	`CJ_9_var_a`	definitional
CJ_9_var_b: $O \Rightarrow \Box_p O$	`CJ_9_var_b`	definitional
CJ_10: restriction under possibility witness	`CJ_10`	`sem_5e`
CJ_11a_var: conjunction of actual obligations	`CJ_11a_var`	`sem_5c, sem_3a`
CJ_15: commitment extension	`CJ_15`	`sem_5d, sem_5e`

Remark 3. *CJ_7 and CJ_8 hold under* classical modal validity *but not under the weaker LD validity. The Lean proofs make this explicit: they take a hypothesis* `modal_valid (pequ A B)` *rather than its LD-valid version, matching the standard caveat in Carmo & Jones (2002), §5.*

Additionally, the bridge between conditional, actual, and ideal obligation is established in `DDLPlus/DTSBridge.lean`: `axiomD_actual` shows that actual obligation satisfies the KD axiom (seriality of obligation), and `DDLPlusFrame_to_DTS` extracts a DTS structure from any DDL+ frame.

4 Deontic Traditional Scheme

The Deontic Traditional Scheme (DTS) [1, 17] provides a three-valued classification of propositions: obligatory, permitted, or optional. We formalize it with a single primitive `ob` and derive the rest:

```
structure DTS (P : Type*) where
  ob        : P → Prop          -- obligation (primitive)
  neg       : P → P             -- negation on propositions
  neg_neg   :   ∀   p, neg (neg p) = p
  consistent :   ∀   p, ob p → not (ob (neg p))
```

```
-- Derived modalities
def DTS.pe (d : DTS P) (p : P) : Prop := not (d.ob (d.neg p))
def DTS.op (d : DTS P) (p : P) : Prop :=
  not (d.ob p) /\ not (d.ob (d.neg p))
```

With these definitions, all 12 von Wright interdefinability relations become theorems (Table 2).

Table 2: DTS interdefinability theorems in Lean 4

Statement	**Lean identifier**
$\mathbf{O}(p) \Rightarrow \mathbf{P}(p)$	`ob_implies_pe`
$\mathbf{P}(p) \Leftrightarrow \neg\mathbf{O}(\neg p)$	`pe_iff_not_ob_neg`
$\mathbf{Opt}(p) \Leftrightarrow \neg\mathbf{O}(p) \wedge \neg\mathbf{O}(\neg p)$	`op_iff`
$\neg\mathbf{P}(p) \Rightarrow \mathbf{O}(\neg p)$	`not_pe_implies_ob_neg`
$\neg\mathbf{Opt}(p) \Leftrightarrow \mathbf{O}(p) \vee \mathbf{O}(\neg p)$	`not_op_iff`
$\mathbf{P}(p) \wedge \mathbf{P}(\neg p) \Leftrightarrow \mathbf{Opt}(p)$	`pe_and_pe_neg_iff_op`
$\mathbf{O}(p) \Rightarrow \neg\mathbf{Opt}(p)$	`ob_implies_not_op`
$\mathbf{Opt}(p) \Rightarrow \mathbf{P}(p)$	`pe_of_op`
$\mathbf{Opt}(p) \Rightarrow \mathbf{P}(\neg p)$	`pe_neg_of_op`
Trichotomy: $\mathbf{O}(p) \vee \mathbf{Opt}(p) \vee \mathbf{O}(\neg p)$	`dts_trichotomy`
Exclusion: at most one of the three holds	`dts_exclusive`
$\mathbf{O}(p) \Rightarrow \neg\mathbf{P}(\neg p)$	`ob_implies_not_pe_neg`

Remark 4 (On the nature of DTS theorems)**.** *These interdefinability relations are largely definitional consequences of* `pe` *and* `op` *being defined in terms of* `ob` *and* `neg`*. The genuine content of the* DTS *formalization is not any individual theorem but rather the* structure *itself: that obligation, permission, and optionality form a consistent trichotomy under the consistency axiom* $\forall p, \mathbf{O}(p) \rightarrow \neg\mathbf{O}(\neg p)$*. The definitions do the mathematical work; the theorems confirm the definitions are well-chosen.*

The bridge `DDLPlusFrame_to_DTS` recovers a DTS structure from a DDL+ frame by instantiating `ob` with the frame's dyadic obligation restricted to the universal context and the reflexive world.

Remark 5 (DTS consistency is not free)**.** DTS *consistency (*$\mathbf{O}(p) \rightarrow \neg\mathbf{O}(\neg p)$*) is* not *derivable from the* DDL+ *frame conditions alone. The* DDL+ *axiom* `sem_5a` *prevents* $ob(X, \bot)$ *but does* not *prevent simultaneous* $ob(X, p)$ *and* $ob(X, \neg p)$ *when* $X \cap p \cap \neg p = \emptyset$ *(which it always is). The* `frameToDTS` *bridge therefore requires an explicit consistency hypothesis* `hcons`*, matching the Isabelle development where* DTS *is an independent axiomatisation.*

5 Governance Reasoning Layer

5.1 Hobbs Eventuality Reification

Following Hobbs [10], we treat events, states, and actions as first-class entities:

```
structure Eventuality (Entity Pred : Type*) where
  predicate : Pred                          -- e.g., soaMoor, soaPay
  roles     : ThematicRole → Option Entity -- ISO 24617-4
      assignments
  polarity  : Bool := true                  -- positive or negated
```

The `ThematicRole` type enumerates 15 roles from ISO 24617-4:2014 [11]: agent, patient, theme, experiencer, beneficiary, instrument, location, time, source, goal, cause, result, manner, purpose, condition. Polarity enables negated eventualities without duplicating the type: `Eventuality.negate e` flips `e.polarity`, and `negate_negate` is a theorem (`!(!e) = e`).

5.2 Deontic Query Encoder

The `DeonticQueryEncoder` structure bridges the governance type system and any world-model evidence calculus:

```
structure DeonticQueryEncoder (Entity Pred Query : Type*) where
  modalQuery  : DeonticModality → Eventuality Entity Pred → Query
  groundQuery : CTTriple Entity Pred → Query
```

The five `DeonticModality` constructors are: `rexist` (alethic, Hobbs' "really exists"), `obligatory`, `permitted`, `forbidden` ($\mathbf{O}(\neg p)$), and `optional` ($\neg\mathbf{O}(p) \wedge \neg\mathbf{O}(\neg p)$). The `classify` function maps a proposition to its deontic status: obligatory if $\mathbf{O}(p)$, forbidden if $\mathbf{O}(\neg p)$, optional otherwise. The `modalQuery` field is the key interface: for a given modality and eventuality it produces the WM query whose evidence represents the epistemic degree to which that modality holds.

5.3 Rexist Bridge

The "really exists" (`rexist`) modality implements Hobbs' principle that what really exists actually holds. In the Lean formalization, `RexistBridge` is a *hypothesis*: a `WMQueryEq` assumption stating that rexist-evidence equals ground-evidence for the same predicate. The bridge soundness theorem (`rexistBridgeRule`) simply unpacks this assumption. No modal axioms are needed — the rexist bridge is an interface contract between the governance layer and whatever world-model evidence calculus is plugged in.

5.4 Three-Level Judgment Pipeline

The `Judgments.lean` file defines a three-level inference pipeline:

1. **Eventuality level**: evidence that an eventuality exists with a given modality (`EventualityJudgment`).
2. **Statement level**: lifting eventuality judgments to meta-triples with identity tags (`StatementJudgment`).

3. **Verdict level**: a `GovernanceVerdict` combining multiple statement judgments into an overall governance outcome.

The pipeline mirrors the three reasoning levels of the `governance-reasoning-engine` source [6] (eventuality_level, statement_level, conclusion_level).

5.5 Worked Example: Rental Obligation

We illustrate the full pipeline with a concrete scenario.

Setup. A rental agreement states:

(P) *Primary obligation*: The tenant ought to pay rent. $\mathbf{O}(\text{pay})$

(CTD) *Contrary-to-duty*: If the tenant does not pay, the tenant ought to give notice. $\mathbf{O}(\text{notice} \mid \neg\text{pay})$

This is a classic Chisholm-paradox scenario [5]. DDL+'s dyadic obligation handles it without contradiction because $\mathbf{O}(\text{pay})$ and $\mathbf{O}(\text{notice} \mid \neg\text{pay})$ live at different conditioning contexts.

Level 1: Eventuality judgment. The eventuality "Alice pays Bob rent" is encoded as:

```
def e_pay : Eventuality Entity Pred :=
  { predicate := soaPay,
    roles := fun
      | .agent => some Alice
      | .patient => some Bob
      | .theme => some Rent
      | _ => none,
    polarity := true }
```

A world-model query `modalQuery .obligatory e_pay` returns evidence $\langle s, n\rangle$ (e.g., $s = 5$ positive observations out of $n = 7$ total), yielding:

```
EventualityJudgment .obligatory e_pay evidence
```

Level 2: Statement judgment. The eventuality judgment is lifted to a tagged statement referencing the lease clause:

```
StatementJudgment "lease_clause_3" eventualityJudgment
```

Level 3: Governance verdict. The verdict engine checks:

- $\mathbf{O}(\text{pay})$ holds (evidence above threshold).
- $\mathbf{P}(\text{pay})$ holds (from `ob_implies_pe`).

- $\neg\mathbf{O}(\neg\text{pay})$ holds (DTS consistency: the agreement does not also oblige non-payment).
- No conflict detected: the DTS trichotomy classifies the tenant as obligated, not optional or forbidden.

Result: `GovernanceVerdict.compliant`.

If the tenant fails to pay, the CTD obligation $\mathbf{O}(\text{notice} \mid \neg\text{pay})$ activates via the conditional obligation operator (`cond_obl`). The verdict shifts to `violation` for the primary obligation, but the CTD provides a recovery path — exactly the scenario DDL+ was designed to handle.

6 GovNormCycle: Constructive Reactive Accessibility

The `ClosedGovAccessibility` structure requires two proofs: *seriality* (every live state has a successor) and *closure* (successors of live states are live). A naive approach using a terminating language such as PyashCore fails: the liveness predicate "`instr` $\neq$ `Done`" is not forward-closed under reduction, as witnessed by `pyashCore_isGovLive_not_closed`.

The GovNormCycle language. We provide a purpose-built 2-state reactive loop:

```
def govDeliberateState : Pattern := .apply "GovDeliberate" []
def govEnactState      : Pattern := .apply "GovEnact" []

inductive GovNormCycleStep : Pattern → Pattern → Prop where
  | deliberateToEnact : GovNormCycleStep govDeliberateState
      govEnactState
  | enactToDeliberate : GovNormCycleStep govEnactState
      govDeliberateState

def govNormLive : Pattern → Prop :=
  fun p => p = govDeliberateState \/ p = govEnactState
```

Both states are permanently live. Seriality and closure follow directly from the two constructors:

```
theorem govNormCycle_serial :
      ∀   p, govNormLive p →   ∃   q, GovNormCycleStep p q := by
  intro p hp
  rcases hp with rfl | rfl
  . exact ⟨govEnactState, .deliberateToEnact⟩
  . exact ⟨govDeliberateState, .enactToDeliberate⟩

theorem govNormCycle_closed :
      ∀   p q, govNormLive p → GovNormCycleStep p q →
        govNormLive q := by
  intro p q _hp hpq
  cases hpq
```

```
  . right; rfl   -- deliberateToEnact: q = govEnactState
  . left; rfl    -- enactToDeliberate: q = govDeliberateState
```

Theorem 6 (Constructive governance accessibility)**.** *The 2-state deliberate-enact cycle satisfies seriality and closure, providing a constructive witness that* `ClosedGovAccessibility` *is satisfiable: every live state has a successor, and successors of live states are live.*

This concrete instance can be composed with any `GovFrame` to produce a `GovernanceDDLBundle`. The model is deliberately minimal: the two states represent the idealised governance cycle (deliberation alternates with enactment) without encoding any domain-specific policy content.

Remark 7 (Why not PyashCore?)**.** *One might expect to instantiate governance accessibility using a richer operational language such as PyashCore. However, the liveness predicate "*`instr` $\neq$ `Done`*" is* not *forward-closed under reduction: PyashCore programs can reach* `Done`*, violating seriality. This is proven as* `pyashCore_isGovLive_not_closed`*. The* `GovNormCycle` *is therefore the simplest language that satisfies the seriality requirement, serving as a proof of satisfiability while leaving richer instantiations to future work.*

7 Temporal-Deontic Bridge

7.1 Temporal Meanings

To model time-varying obligations we pair a base world type with a time type:

```
abbrev TemporalWorld (w T : Type*) := w x T
abbrev TemporalMeaning (c w T : Type*) := Meaning c (w x T)
```

Lead (future) and lag (past) shifts on meanings capture how propositions vary with time (where `T` has `[Add T]` or `[Sub T]` respectively):

```
-- phi evaluated Delta time units ahead
def meaningLead (phi : TemporalMeaning c w T) (Delta : T) :=
  fun ctx (v, t) => phi ctx (v, t + Delta)

-- phi evaluated Delta time units in the past
def meaningLag (phi : TemporalMeaning c w T) (Delta : T) :=
  fun ctx (v, t) => phi ctx (v, t - Delta)
```

7.2 Temporally Uniform Obligation

A `DDLPlusFrame` on $w \times T$ is *temporally uniform* if the accessibility and obligation relations are stable under time translation:

```
structure TemporallyUniformOb {w T : Type*} [AddCommGroup T]
    (F : DDLPlusFrame (w x T)) where
  -- pv is translation-invariant
  pv_shift :   ∀   (Delta : T) (v : w) (t : T) (v' : w) (t' : T),
```

```
    F.pv (v, t) (v', t')  ↔  F.pv (v, t + Delta) (v', t' + Delta)
  -- ob is compatible with base-point shift (content lags by Delta)
  ob_shift :   ∀   (Delta : T) (v : w) (t : T) (Y : WProp (w x T))
    ,
    F.ob (F.pv (v, t)) Y  ↔
      F.ob (F.pv (v, t + Delta)) (fun p => Y (p.1, p.2 - Delta))
```

Any concrete temporal frame must discharge these shift-invariance conditions.

7.3 Obligation Persistence Theorem

Theorem 8 (Temporal-lag persistence of ideal obligation)**.** *Let F be a* `DDLPlusFrame` *on $w \times T$ where T is an abelian group, and let* `tub : TemporallyUniformOb F`*. If φ is ideally obligatory at world (v, t_0), then the lag-shifted version $\varphi[\cdot - \Delta]$ is ideally obligatory at $(v, t_0 + \Delta)$.*

Significance. This theorem captures the intuition that obligations persist under temporal perspective shifts. For example, if a tenant is obligated to pay rent on the 1st of the month, then from the perspective of the 15th the obligation to *have paid* 14 days ago still holds. More precisely: obligations do not disappear when one changes temporal reference frame — they shift with it, with the content lagging by the same amount as the reference point advances.
Proof sketch. The proof has three parts:

1. *ob part*: `ob_shift` maps $F.\mathrm{ob}(F.\mathrm{pv}\,(v, t_0),\ \varphi)$ to $F.\mathrm{ob}(F.\mathrm{pv}\,(v, t_0{+}\Delta),\ \lambda p.\, \varphi(p_1, p_2{-}\Delta))$, which is exactly `meaningLag phi Delta`.

2. *pv part*: the violation witness (v_w, t_w) is shifted to $(v_w, t_w + \Delta)$ using `pv_shift`.

3. *Negation*: at the shifted witness, `add_sub_cancel_right` reduces the lag-shift evaluation back to the original negation. □

Remark 9. *The "lag" direction is the only direction provable from* `ob_shift` *as stated: the frame does not directly relate ob at two different base times without the content shift. Shifting the reference time forward requires adjusting all content backward to keep the same relative obligations.*

7.4 Causal-Normative Prohibition

We connect the temporal-deontic bridge to PLN's causal heuristics [8]. A *spurious causal candidate* is an element x at time t with extensional but not intensional predictive support:

```
structure SpuriousCausalCandidate
    (profile : PredictiveCausalProfile Domain Time)
    (x : Domain) (t : Time) : Prop where
  extensional    : profile.extensional x t
  not_intensional : not (profile.intensional x t)
```

Theorem 10 (Spurious prohibition)**.** *If a* DTS *obligation structure obliges only intensionally-grounded elements (`ObligesOnlyIntensional`), then no spurious causal candidate can be obligated.*

The proof is a one-line contradiction: `hSpur.not_intensional` contradicts `hGnd.ob_intensional x hOb t`. The contrapositive (`obligatory_not_spurious`) is derived immediately.

8 Abstract Modal Query Encoder

The event-calculus [13, 16] and the governance reasoning layer both define query encoders of the form `Modality -> Referent -> Query`, but they were independently defined. We introduce an abstract structure that unifies both:

```
structure ModalQueryEncoder (Modality Referent Query : Type*) where
  encode : Modality → Referent → Query
```

Event-calculus instance. The event calculus uses three temporal modalities:

```
inductive EventModality where
  | holdsAt      -- fluent holds at time t
  | initiatedAt  -- fluent initiated at t
  | terminatedAt -- fluent terminated at t
```

An `EventQueryEncoder Event Time Query` (with fields `holdsAt`, `initiatedAt`, `terminatedAt`) is equivalent to a `ModalQueryEncoder EventModality (Event x Time) Query`:

```
def eventQueryEncoderToModal (enc : EventQueryEncoder Event Time
    Query) :
    ModalQueryEncoder EventModality (Event x Time) Query where
  encode
    | .holdsAt,      (e, t) => enc.holdsAt e t
    | .initiatedAt,  (e, t) => enc.initiatedAt e t
    | .terminatedAt, (e, t) => enc.terminatedAt e t
```

Theorem 11 (Event-encoder roundtrip)**.** *The maps `eventQueryEncoderToModal` and `modalToEventQueryEncoder` are mutually inverse. Both roundtrips are proven as simp lemmas using `cases modality` and `rfl`.*

Deontic instance. The `DeonticQueryEncoder.modalQuery` field has exactly the shape `DeonticModality -> Eventuality Entity Pred -> Query`, giving:

```
def deonticQueryEncoderToModal (d : DeonticQueryEncoder Entity Pred
     Query) :
    ModalQueryEncoder DeonticModality (Eventuality Entity Pred)
        Query where
  encode := d.modalQuery
```

The embedding is a direct field projection. With this unification, dispatch infrastructure, traversal utilities, and test harnesses written for `ModalQueryEncoder` apply to both the event-calculus and deontic encoders without duplication.

9 Gewirth's Principle of Generic Consistency

The Principle of Generic Consistency (PGC) [7] asserts:

> *Act in accord with the generic rights of your recipients as well as of yourself.*

Gewirth's argument proceeds from agency: an agent who acts on purpose must claim the necessary conditions for agency (freedom and well-being, *FWB*) as rights, and by universalizability must grant the same rights to all agents.

Benzmüller et al. [3] gave the first machine-checked formalization of the PGC in Isabelle/HOL, available in the Archive of Formal Proofs. Our Lean 4 port follows their proof structure while connecting it to the DDL+ and DTS layers developed in the preceding sections.

The formalization in `Mettapedia/Ethics/` uses the DDL+ embedding developed above:

- `Core.lean`: introduces the agent world type, purpose-directed action predicate `PPA`, and the generic rights type `FWBRight`.
- `GewirthPGC.lean`: packages a `PGCInterpretation` (frame + assignments) and states the main theorem: `PGC_strong_ofAssumptions` proves that any agent satisfying `PGCAssumptions` is ideally obligated to respect the FWB rights of all purposive agents.
- `GewirthBridge.lean`: connects the local `Oi` and `Ocond` operators in `GewirthPGC.lean` to the corresponding `ideal_obl` and `cond_obl` from `DDLPlus.Core`, making the bridge explicit.

The DTS interdefinability theorems (§4) underlie the Gewirth formalization: the equivalences between obligation, permission, and optionality ensure that the logical structure of the PGC proof is sound regardless of which representation is chosen.

10 MeTTa-to-Lean Refinement Theorems

The `governance-reasoning-engine` [6] implements DTS rules as MeTTa rewrite rules. A natural question is whether these operational rules *correctly encode* the DTS theorems proven semantically in Lean. This section presents formal refinement theorems answering this question for three evaluator levels: PeTTa (type-free), HE MeTTa (with type and binding threading), and an answer-level projection that abstracts over both.

10.1 Architecture

The refinement bridge has four layers:

1. **Rule encoding.** Each MeTTa governance rule (e.g., `(= (ct-triple-for-add $e type permitted) (ct-triple $e type obligatory))`) is encoded as a concrete Lean `RewriteRule` value with `premises = []`.
2. **Deontic interpretation.** A function `interpretDeontic : Pattern -> DeonticInterp` classifies ground patterns into deontic assertions: `ct-triple` patterns map to `.modalAssertion e .obligatory`, `ct-triple-for-add` patterns to `.modalAssertion e .permitted`, and so on for `.forbidden` and `.optional`.
3. **Interpretation correctness.** For each rule, we prove that the left-hand side and right-hand side, after binding substitution, interpret to the expected deontic modalities.
4. **Semantic refinement.** We bundle the interpretation correctness proofs with the corresponding DTS theorem, showing that the rule structure matches the proven algebraic property.

10.2 PeTTa Refinement

PeTTa is the type-free, binding-free evaluator: `PeTTaEval s p [q]` states that pattern `p` evaluates to `[q]` in space `s` via rewrite rule application.

Theorem 12 (PeTTa OB⇒PE refinement)**.** *For any string* `e`*:*

1. *interpretDeontic*(*applyBindings* [(*"e"*, *e*)] *rule.right*) = *modalAssertion e obligatory,*
2. *interpretDeontic*(*applyBindings* [(*"e"*, *e*)] *rule.left*) = *modalAssertion e permitted, and*
3. $\forall\,(d : \mathrm{DTS}\ P)\ (p : P),\ d.ob\ p \to d.pe\ p.$

Parts 1 and 2 are proved by `simp` on the concrete rule encoding and binding substitution. Part 3 is the already-proven DTS theorem `ob_implies_pe`. The three-part conjunction bundles rule-level pattern interpretation with the semantic theorem it encodes.

Analogous refinement theorems are proven for $\mathbf{O}(\neg p) \Rightarrow \neg\mathbf{P}(p)$ (`dts_ob_neg_not_pe_refinement`), $\neg\mathbf{P}(p) \Rightarrow \mathbf{O}(\neg p)$, $\mathbf{Opt}(p) \Leftrightarrow \neg\mathbf{O}(p) \wedge \neg\mathbf{O}(\neg p)$, $\mathbf{O}(p) \Rightarrow \neg\mathbf{Opt}(p)$, and the DTS trichotomy — all 7 core interdefinability relations from Table 2.

10.3 Shared MeTTaLike Interface and Let* Equivalence

The MeTTa `let*` construct is not a built-in primitive but a pair of rewrite rules (`letStarBaseRule`, `letStarRecRule`) that unfold `let*` to nested `let` expressions. To prove these unfoldings generically, we introduce a typeclass:

```
class MeTTaLike (Eval : PeTTaSpace → Pattern → List Pattern →
    Prop) where
  ruleApp :   ∀   {s r bs p},
    r in s.rules → r.premises = [] → bs in matchPattern r.left p
        →
    Eval s p [applyBindings bs r.right]
```

Both `PeTTaEval` (the type-free evaluator) and `HEEvalAnswers` (the answer-level projection of HE MeTTa) are instances. With this abstraction, unfolding theorems hold for *any* MeTTaLike evaluator:

Theorem 13 (Let* unfolding)**.** *For any MeTTaLike evaluator* `Eval` *and space* `s` *containing the* `let*` *rules:*

$$\mathit{Eval}\ s\ (\mathit{let*}\ [(v_1, e_1), (v_2, e_2)]\ \mathit{body})\ [\mathit{let}\ v_1\ e_1\ (\mathit{let*}\ [(v_2, e_2)]\ \mathit{body})]$$
$$\mathit{Eval}\ s\ (\mathit{let*}\ [(v_2, e_2)]\ \mathit{body})\ [\mathit{let}\ v_2\ e_2\ (\mathit{let*}\ []\ \mathit{body})]$$
$$\mathit{Eval}\ s\ (\mathit{let*}\ []\ \mathit{body})\ [\mathit{body}]$$

The match proofs use `simp` on the concrete `matchBag` semantics for bag patterns with rest variables. The full unfolding sequence for 2-binding and 3-binding `let*` chains is bundled as `letStar_full_2` and `letStar_full_3`. This is significant because the DTS governance rules in `DTS.metta` (lines 14–135) use `let*` chains extensively; the unfolding theorems ensure these chains reduce correctly under any compliant evaluator.

10.4 HE MeTTa Refinement

The HE (Hyperon Experimental) MeTTa evaluator extends PeTTa with explicit type annotations and binding threading: `MeTTaEval s p ty inputBindings results` produces result pairs `(pattern, bindings)`. The answer-level projection erases types and bindings:

```
def HEEvalAnswers (s : PeTTaSpace) (p : Pattern)
    (answers : List Pattern) : Prop :=
    ∃   ty inputBs results,
    MeTTaEval s p ty inputBs results
    /\ results.map Prod.fst = answers
```

All PeTTa refinement theorems lift automatically to `HEEvalAnswers` via the shared `MeTTaLike` instance. Additionally, full `MeTTaEval`-level theorems are proven with explicit binding tracking:

Theorem 14 (HE binding threading for OB⇒PE)**.** *If* `dtsRule_ob_implies_pe` *is in a PeTTa space, the HE MeTTa evaluator fires it with match bindings* `[("e", .apply e [])]` *prepended to the input bindings, and the interpretation/semantic refinement hold as before.*

10.5 Canonical HE Spec Architecture

The Lean formalization contains two representations of HE MeTTa semantics:

- **Computable interpreter**: `Languages/MeTTa/HE/Interpreter.lean` defines 6 mutually recursive functions (`metta`, `interpretExpression`, `interpretFunction`, `interpretArgs`, `interpretTuple`, `mettaCall`) with fuel-bounded execution, matching the Rust reference implementation. A suite of 37 conformance theorems (`HE/Conformance.lean`) verifies this interpreter against the spec.

- **Inductive relation**: `OSLF/PeTTa/MeTTaEval.lean` provides the same semantics as an inductive proposition, amenable to proof-theoretic reasoning. The refinement theorems in this section use this representation.

Bridging the computable interpreter to the inductive relation requires reconciling different type representations (`Atom` with 4 constructors vs. `Pattern` with 7 constructors) and different binding models (`HE.Bindings` with assignments and equalities vs. simple association lists). A lossy bridge (`patternToAtom`/`atomToPattern`) exists in `MeTTaCore/Bridge.lean`; a full general equivalence proof remains future work, but the DTS rule subset is *exact*: governance rules use only `apply` and `fvar` pattern constructors, which map cleanly to `expression` and `var` atoms.

For this exact subset, `HECanonicalConformance.lean` proves that the canonical HE interpreter produces the correct results via kernel-checked `rfl` proofs:

```
theorem he_canonical_ob_pe_soaMoor :
    eval (expression [symbol "ct-triple-for-add",
            expression [symbol "soaMoor"], ...])
         dtsHESpace =
    [(expression [symbol "ct-triple",
            expression [symbol "soaMoor"], ...],
      { assignments := [("e", ...)], equalities := [] })] := rfl
```

The Lean kernel itself performs the full HE evaluation (through `metta` → `interpretExpression` → `mettaCall` → `queryEquations` → `simpleMatch` → `Bindings.apply`) and verifies the output. End-to-end conformance bundles eval, decode (`atomToPattern`), deontic interpretation, and the DTS theorem into a single conjunction — all kernel-checked.

10.6 What the Refinement Proves

The refinement theorems establish:

> *If the governance-reasoning-engine DTS rules are encoded as PeTTa rewrite rules, then evaluating them produces results that, under deontic interpretation, satisfy the corresponding* DTS *theorems proven in Lean.*

This is a soundness result at three levels:

1. **Inductive semantics**: The `PeTTaEval` and `MeTTaEval` relations fire the DTS rules with correct binding substitution and deontic interpretation (7 rules each).

2. **Shared evaluator abstraction**: The `MeTTaLike` typeclass ensures `let*` unfolding works identically for both evaluators (2- and 3-binding chains proven).

3. **Canonical interpreter**: The computable HE `eval` function produces the correct results for all three DTS rules, verified by `rfl` (the Lean kernel performs the computation).

The verification does not cover the full MeTTa codebase, but it does verify that the core deontic rewrite rules faithfully encode the algebraic properties they claim to implement, and that the canonical interpreter agrees with the inductive semantics on the DTS rule subset.

All refinement proofs are kernel-checked with zero `sorry` placeholders and use only the standard axioms (`propext`, `Classical.choice`, `Quot.sound`).

11 Related Work

Isabelle/HOL formalizations. Benzmüller et al. [2] formalized DDL+ in Isabelle/HOL using the same shallow semantic embedding approach; our Lean 4 DDL+ formalization is a port of that work. Benzmüller and Fuenmayor [3] formalized the Gewirth PGC in Isabelle/HOL, available in the Archive of Formal Proofs; our PGC formalization is likewise a port. We extend both with the governance reasoning layer, temporal bridges, and PLN causal integration. The Isabelle sources (`CJDDLplus.thy`, `ExtendedDDL.thy`) are included in the repository as a reference.

Input/Output logic. Makinson and van der Torre's input/output logic [14] provides an alternative semantics for conditional obligation. The `ob` dyadic relation in DDL+ can be interpreted in their framework, though the present formalization does not develop this connection explicitly.

MeTTa governance reasoning engine. The `governance-reasoning-engine` [6] implements a three-level inference pipeline (eventuality, statement, verdict) in MeTTa, using non-deterministic evaluation for governance verdict computation. Our formalization verifies the algebraic properties (DTS interdefinability, evidence monotonicity) that correspond to structures used in their implementation, and the refinement theorems of §10 formally connect the MeTTa rule encodings to the Lean semantics for all 7 core DTS interdefinability relations.

Event calculus in Lean. The PLN Probabilistic Event Calculus formalized in `PLNProbabilisticEventCalculus.lean` (this repository) follows the Kowalski-Sergot [13] and Shanahan [16] presentations. The `ModalQueryEncoder` of §8 unifies this with the deontic layer.

12 Limitations

- **No end-to-end regulatory corpus.** The formalization provides verified logical primitives but has not been tested against any regulatory corpus (GDPR, SEC rules, etc.).
- **No natural language parser.** There is no automated pipeline from regulatory text to DDL+ frame instances; all examples are hand-encoded.

- **No benchmark suite.** We do not provide a test suite of governance scenarios with expected verdicts against which implementations could be evaluated.
- **Rule-level (not full-program) refinement.** The refinement theorems (§10) verify that individual DTS rewrite rules correctly encode the corresponding DTS theorems, but they do not verify the full MeTTa governance-reasoning-engine codebase. The three-level inference pipeline (eventuality, statement, verdict) is verified at the rule-firing level, not end-to-end. The canonical HE interpreter and the inductive `MeTTaEval` relation agree on the DTS rule subset (proven via `rfl` conformance tests), but a general equivalence has not been established.
- **No deployed case study.** The system has not been deployed in any real governance or compliance setting.
- **DTS consistency requires external hypothesis.** The DTS consistency axiom ($\mathbf{O}(p) \to \neg\mathbf{O}(\neg p)$) is not derivable from DDL+ frame conditions alone; the bridge `frameToDTS` requires an explicit `hcons` parameter (see Remark in §4).
- **Minimal accessibility model.** The `GovNormCycle` is a 2-state proof of satisfiability, not a practical governance state machine.

13 Future Developments: Toward Practical Governance

The formalization presented here establishes a verified logical foundation but does not yet constitute a deployable governance tool. Several real-world systems demonstrate that deontic logic *can* deliver practical value, and point toward concrete development paths.

Regulatory compliance checking. Governatori and Shek's *Regorous* system [9] uses defeasible deontic logic to check business processes (modelled in BPMN) against regulatory requirements, including GDPR formalization. A natural extension of our work would be to compile DDL+ frame instances from regulatory texts (using NLP extraction or GF grammars) and check process traces against the resulting obligation/permission structure. The DTS trichotomy (obligatory/optional/forbidden) maps directly to compliance verdicts.

Multi-agent governance for AI systems. XMPro's Multi-Agent Generative Systems (MAGS) platform [18] uses deontic principles as "Rules of Engagement" for industrial AI agents: obligations that agents must fulfil, permissions that bound their autonomy, and prohibitions they cannot violate. These are encoded as *deontic tokens* embedded in agent profiles. Our DDL+ formalization could provide verified backing for such tokens: each deontic token would carry

a proof witness (from the CJ theorems) ensuring it is consistent with the frame conditions, ruling out contradictory obligations before deployment.

Contract clause conflict detection. Recent legal AI systems [15] use embedding-based similarity to detect conflicting clauses across contracts and regulatory documents. These systems achieve high recall but lack formal guarantees. A hybrid approach would use NLP to extract candidate norm pairs, then apply DDL+ conflict detection (via `sem_5a`: no obligation can require the impossible, and `sem_5c`: consistent obligations are conjoinable) to formally verify whether detected conflicts are genuine.

Contrary-to-duty policy repair. The Chisholm paradox (§5.5) shows that naive deontic logic produces contradictions when primary obligations are violated. DDL+'s dyadic obligation resolves this, but the formalization does not yet support automated *repair*: given a detected violation, synthesize the applicable contrary-to-duty obligations and present them as a recovery plan. This would require integrating the temporal bridge (§7) with a planning layer that enumerates CTD chains.

Richer governance accessibility models. The `GovNormCycle` (§6) is a minimal 2-state witness of satisfiability. A practical system needs governance accessibility over richer state spaces — for instance, a reactive event-loop language where each "state" is a snapshot of active norms, pending obligations, and violation history. The negative result for PyashCore (`pyashCore_isGovLive_not_closed`) shows that terminating languages fail the seriality requirement; future work should explore co-inductive or stream-based state machines that guarantee perpetual liveness.

Probabilistic evidence integration. The three-level judgment pipeline (§5) uses PLN evidence values but does not yet connect them to posterior-mean inference. Integrating the DTS strength values with the probabilistic event calculus (via the `ModalQueryEncoder`) would allow governance verdicts to carry calibrated confidence levels, enabling graded compliance rather than binary pass/-fail.

14 Conclusion

We have presented a Lean 4 formalization of governance deontic logic spanning six layers: DDL+ semantic embedding, DTS algebra, Hobbs eventuality reification with governance judgments, constructive reactive accessibility, temporal-deontic and causal-normative bridges, the Gewirth PGC, and formal MeTTa-to-Lean refinement theorems.

The refinement chain is the deepest contribution: it connects operational MeTTa governance rules to proven DTS theorems at three levels — inductive

semantics (PeTTa and HE MeTTa), a shared evaluator abstraction with `let*` unfolding, and kernel-checked canonical interpreter conformance. For the DTS rule subset, the Lean kernel itself performs the HE evaluation and verifies the output.

The formalization is compositional (each layer is independently usable) and grounded in published sources (Carmo & Jones 2002, von Wright 1951, Hobbs 1985, ISO 24617-4).

References

[1] Carlos E. Alchourrón and Eugenio Bulygin. *Normative Systems*. Springer, 1971.

[2] Christoph Benzüller, Ali Farjami, and Xavier Parent. A dyadic deontic logic in HOL. In *Deontic Logic and Normative Systems*, volume 10833 of *Lecture Notes in Artificial Intelligence*, pages 33–49. Springer, 2018.

[3] Christoph Benzüller and David Fuenmayor. Formalisation and evaluation of Alan Gewirth's proof for the Principle of Generic Consistency in Isabelle/HOL. In *Archive of Formal Proofs*, 2018. `https://www.isa-afp.org/entries/GewirthPGCProof.html`.

[4] José Carmo and Andrew J.I. Jones. Deontic logic and contrary-to-duties. In *Handbook of Philosophical Logic*, volume 8, pages 265–343. Kluwer, 2nd edition, 2002.

[5] Roderick M. Chisholm. Contrary-to-duty imperatives and deontic logic. *Analysis*, 24(2):33–36, 1963.

[6] Formal Methods Group. governance-reasoning-engine: A MeTTa implementation of governance reasoning. `https://github.com/Formal-Methods-Group/governance-reasoning-engine`, 2025.

[7] Alan Gewirth. *Reason and Morality*. University of Chicago Press, 1978.

[8] Ben Goertzel, Matthew Iklé, Izabela Freire Goertzel, and Ari Heljakka. *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*. Springer, 2008.

[9] Guido Governatori and Sidney Shek. Regorous: A business process compliance checker. In *Proceedings of the 14th International Conference on Artificial Intelligence and Law (ICAIL)*, pages 245–246. ACM, 2013.

[10] Jerry R. Hobbs. Ontological promiscuity. In *Proceedings of the 23rd Annual Meeting of the Association for Computational Linguistics*, pages 61–69, 1985.

[11] Language resource management — semantic annotation framework (SemAF) — part 4: Semantic roles. ISO Standard 24617-4, 2014.

[12] David Kaplan. Demonstratives. In Joseph Almog, John Perry, and Howard Wettstein, editors, *Themes from Kaplan*, pages 481–563. Oxford University Press, 1989.

[13] Robert Kowalski and Marek Sergot. A logic-based calculus of events. *New Generation Computing*, 4(1):67–95, 1986.

[14] David Makinson and Leendert van der Torre. Input/output logics. In *Journal of Philosophical Logic*, volume 29, pages 383–408, 2000.

[15] Norm AI. Legal engineering: AI-powered regulatory compliance. `https://www.norm.ai/`, 2025.

[16] Murray Shanahan. The event calculus explained. In *Artificial Intelligence Today*, volume 1600 of *Lecture Notes in Computer Science*, pages 409–430. Springer, 1999.

[17] Georg Henrik von Wright. Deontic logic. *Mind*, 60(237):1–15, 1951.

[18] XMPro. Multi-agent generative systems: Deontic principles for industrial AI governance. `https://github.com/XMPro/Multi-Agent/blob/main/docs/concepts/deontic-principles.md`, 2025.